

第一章 绪论

1. 数据库的4个基本概念

- 数据(Data)
- 数据库(Database):

数据库（Database，简称DB）是长期储存在计算机内、有组织的、可共享的大量数据的集合

- 数据库管理系统(DBMS)
- 数据库系统(DBS)

数据库系统的特点

- 数据结构化
- 数据的共享性高，冗余度低且易扩充
- 数据独立性高
- 数据由数据库管理系统统一管理和控制

2. 数据库管理系统

- 什么是数据库管理系统
 1. 位于用户与操作系统之间的一层数据管理软件
 2. 是基础软件，是一个大型复杂的软件系统

数据库管理系统的用途

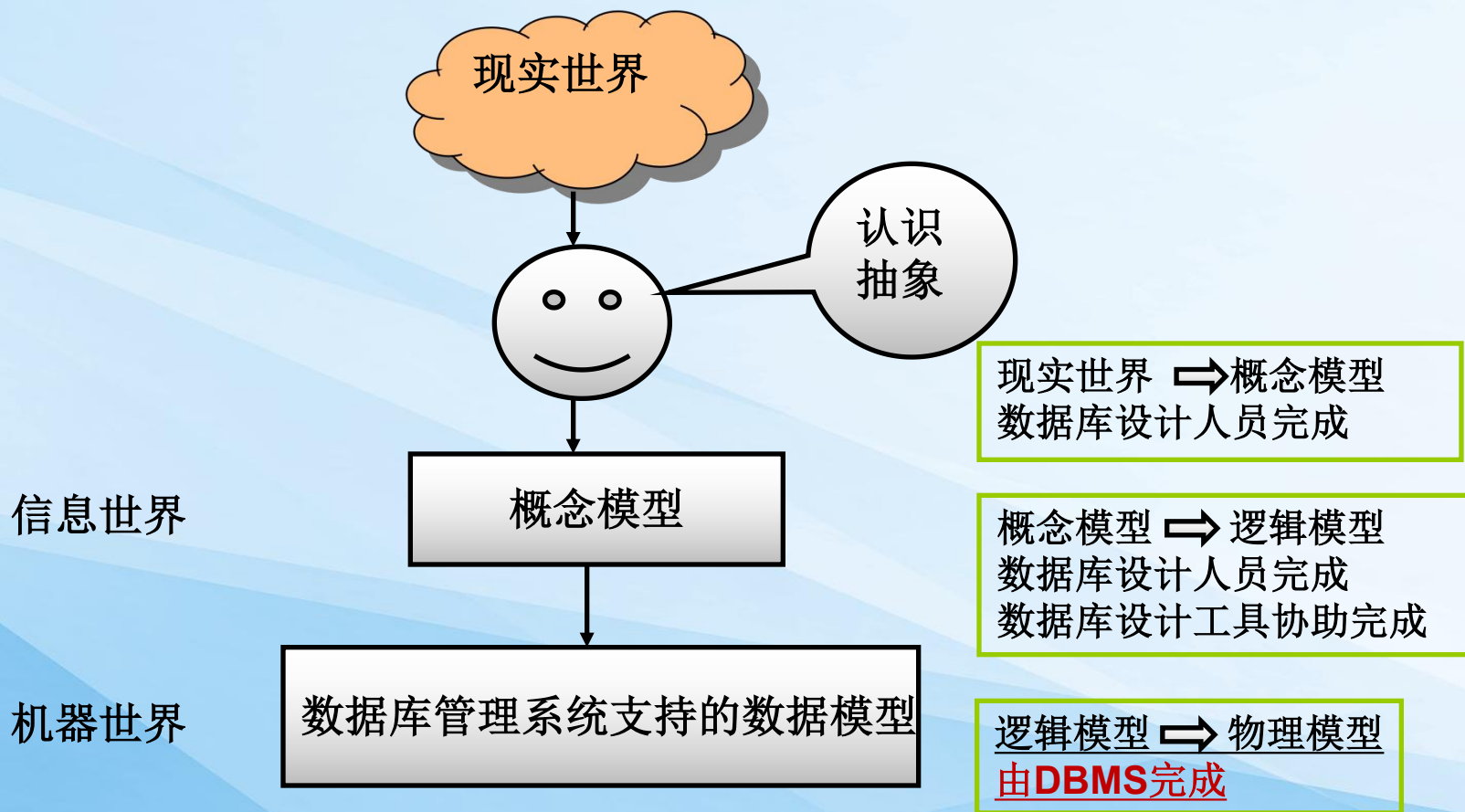
科学地组织和存储数据、高效地获取和维护数据

数据库在建立、使用和维护时由**DBMS**统一管理、统一控制。

3. 数据模型

- 数据模型是对现实世界数据特征的抽象。
- 数据模型分为两类（两个不同的层次）
 - （1）**概念模型** 也称信息模型，它是按用户的观点来对数据和信息建模，用于数据库设计。不依赖于具体的计算机系统，即与**DBMS**无关。
 - （2）**逻辑模型和物理模型**
 - 逻辑模型主要包括网状模型、层次模型、**关系模型**、面向对象数据模型、对象关系数据模型、半结构化数据模型等。按计算机系统的观点对数据建模，用于**DBMS**实现。
 - 物理模型是对数据最底层的抽象，描述数据在系统内部的表示方式和存取方法，在磁盘或磁带上的存储方式和存取方法。

要求会设计E-R图，会向逻辑模型进行转化，即转换成关系模式



现实世界中客观对象的抽象过程

数据模型的组成要素

数据模型精确地描述了系统的静态特性、动态特性和完整性约束条件。

- 数据结构：描述系统的静态特性
- 数据操作：描述系统的动态特性
- 数据的完整性约束条件

4. 数据库系统的三级模式结构

1.从数据库**管理系统角度**看，数据库系统通常采用三级模式结构，是数据库系统内部的系统结构：

内模式、模式、外模式

2.二级映象在**DBMS**内部实现这三个抽象层次的联系和转换

1) 外模式 / 模式映像； 2) 模式 / 内模式映像

3.数据库的数据独立性指逻辑独立性和物理独立性，请问如何实现？

4. 这两层映象保证了数据库系统中的数据能够具有较高的逻辑独立性和物理独立性

数据库系统的三级模式结构（续）

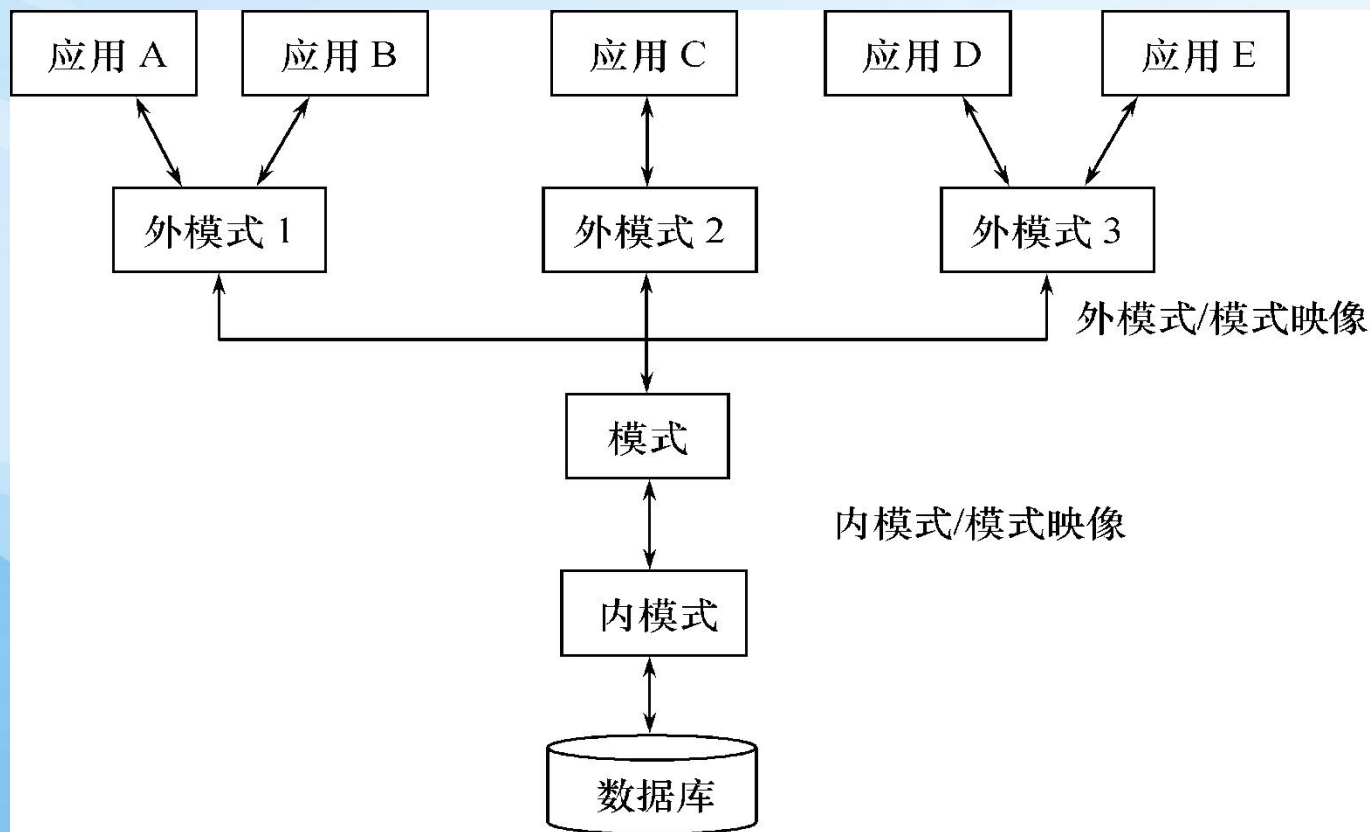
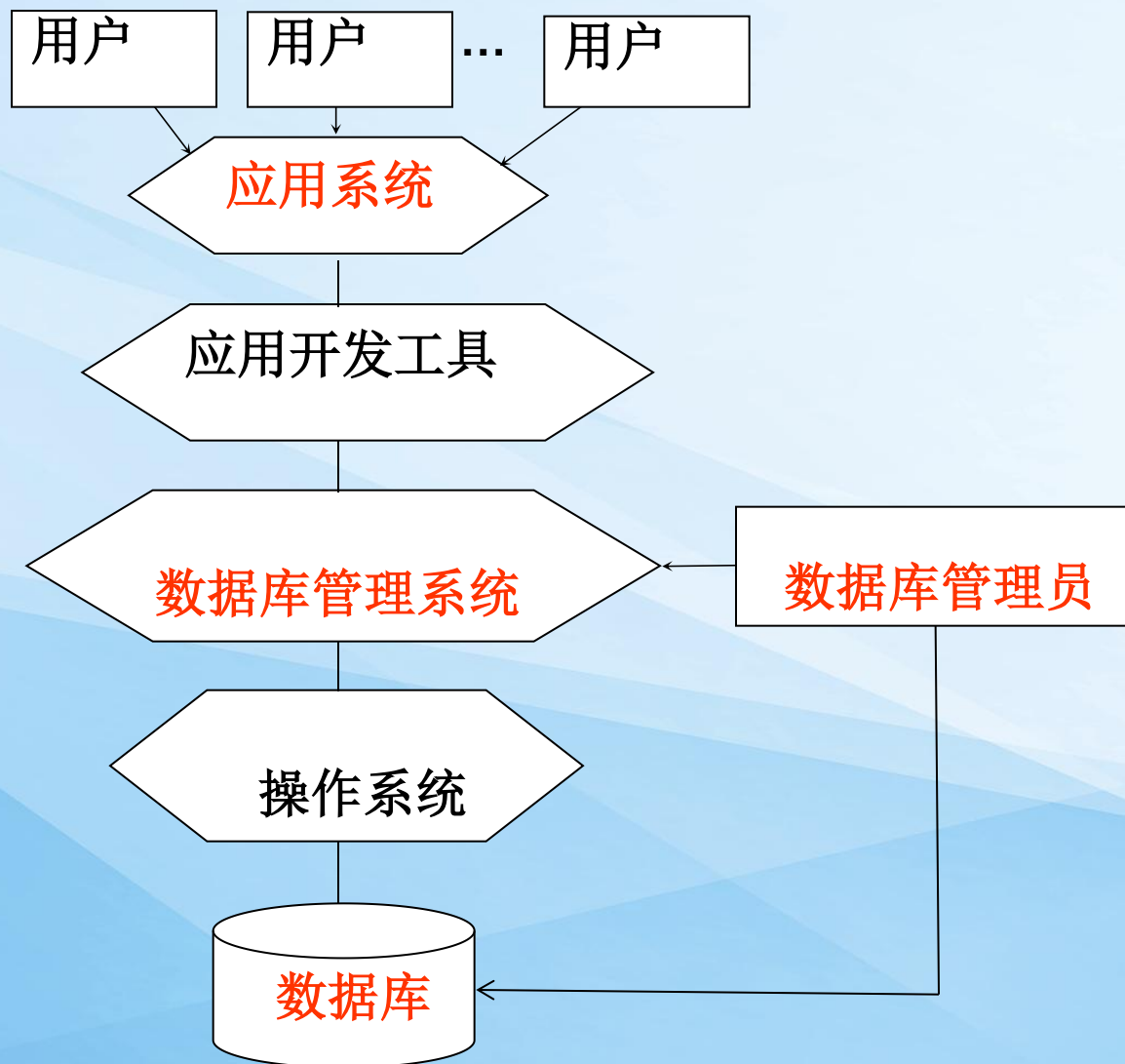


图1.16 数据库系统的三级模式结构

5. 数据库系统(DBS)的组成

- 数据库**DB**
- 数据库管理系统（及其开发工具）**DBMS**
- 应用系统
- 数据库管理员（**DBA**）



第二章 关系数据库

- 关系数据库的单一的数据结构----关系

现实世界的实体以及实体间的各种联系均用关系来表示

- 逻辑结构----二维表

从用户角度，关系模型中数据的逻辑结构是一张二维表

- 建立在集合代数的基础上

基本关系的性质（续）

基本关系的性质

- ① 列是同质的（Homogeneous）
- ② 不同的列可出自同一个域
 - 其中的每一列称为一个属性
 - 不同的属性要给予不同的属性名
- ③ 列的顺序无所谓，列的次序可以任意交换
- ④ 任意两个元组的候选码不能相同
- ⑤ 行的顺序无所谓，行的次序可以任意交换

基本关系的性质（续）

- ⑥ 分量必须取原子值(关系的每一个分量必须是一个不可分的数据项)

这是规范条件中最基本的一条

表2.3 非规范化关系

SUPERVISOR	SPECIALITY	POSTGRADUATE	
		PG1	PG2
张清玫	计算机专业	李勇	刘晨
刘逸	信息专业	王敏	

小表

关系数据库

一些基本概念：

域、笛卡尔积、元组、分量、基数、属性

码：候选码、全码、主码、超键码

主属性、非主属性

基本的关系操作

- 常用的关系操作
 - 查询操作：选择、投影、连接、除、并、差、交、笛卡尔积
 - 选择、投影、并、差、笛卡尔基是**5**种基本操作
 - 数据更新：插入、删除、修改

关系代数

表2.4 关系代数运算符

运 算 符		含 义
集合运算符	$\cup$	并
	$-$	差
	$\cap$	交
	$\times$	笛卡尔积
专门的关系运算符	σ	选择
	π	投影
		连接
	$\div$	除

关系代数

运算符		含义
比较运算符	$>$	大于
	$\geq$	大于等于
	$<$	小于
	$\leq$	小于等于
	$=$	等于
	$\neq$	不等于

运算符	含义	
逻辑运算符	$\neg$	非
	$\wedge$	与
	$\vee$	或

比较运算符和逻辑运算符是用来辅助**专门的关系运算符**进行操作的。

第三章 关系数据库标准语言SQL

- SQL（Structured Query Language）：结构化查询语言，是关系数据库的标准语言

- SQL的特点

- 1.综合统一

- 2.高度非过程化

3. 面向集合的操作方式

- 4.以同一种语法结构提供两种使用方式

- 5.语言简捷，易学易用

数据定义

基本表的定义、删除与修改

索引的建立与删除

视图的建立与删除

数据查询

SELECT [ALL|DISTINCT] <目标列表表达式> [别名] [,<目标列表表达式> [别名]] ...

FROM <表名或视图名> [别名] [,<表名或视图名> [别名]] ... | (<SELECT语句>)[AS] <别名>

[WHERE <条件表达式>]

[GROUP BY <列名1> [HAVING <条件表达式>]]

[ORDER BY <列名2> [ASC|DESC]]

LIMIT <行数1>[OFFSET <行数2>];

单表查询

- 查询仅涉及一个表：
 - 一、选择表中的若干列
 - 二、选择表中的若干元组
 - 三、**ORDER BY**子句
 - 四、聚集函数
 - 五、**GROUP BY**子句
 - 六、**LIMIT**子句

多表查询

- 一、等值与非等值连接查询
- 二、自身连接
- 三、外连接
- 四、复合条件连接

嵌套查询

1. 带有IN谓词的子查询
2. 带有比较运算符的子查询
3. 带有ANY（SOME）或ALL谓词的子查询
4. 带有EXISTS谓词的子查询

集合查询

1. 并操作UNION
2. 交操作INTERSECT
3. 差操作EXCEPT

数据更新

- 插入数据：插入元组、插入子查询结果
- 修改数据
- 删除数据

第四章 数据库安全性

- 数据库的安全性是指保护数据库以防止不合法使用所造成的数据泄露、更改或破坏。
- 数据库安全性控制的常用方法
 - 用户标识和鉴别
 - 存取控制
 - 视图
 - 审计
 - 数据加密

存取控制

- 常用存取控制方法
 - 自主存取控制（Discretionary Access Control，简称DAC）
 - C2级
 - 用户对不同的数据对象有不同的存取权限
 - 不同的用户对同一对象也有不同的权限
 - 用户还可将其拥有的存取权限转授给其他用户
 - 强制存取控制（Mandatory Access Control，简称 MAC）
 - B1级
 - 每一个数据对象被标以一定的密级
 - 每一个用户也被授予某一个级别的许可证
 - 对于任意一个对象，只有具有合法许可证的用户才可以存取

存取控制

- 常用存取控制方法
 - 自主存取控制
 - 授权与回收: **grant revoke**
 - 数据库角色

存取控制

- 强制存取控制

在强制存取控制中，数据库管理系统所管理的全部实体被分为主体和客体两大类

- 敏感度标记 (Label)

对于主体和客体，DBMS为它们每个实例（值）指派一个敏感度标记 (Label)

- 主体的敏感度标记称为许可证级别 (Clearance Level)

- 客体的敏感度标记称为密级 (Classification Level)

- 强制存取控制规则

(1) 仅当主体的许可证级别大于或等于客体的密级时，该主体才能读取相应的客体

(2) 仅当主体的许可证级别小于或等于客体的密级时，该主体才能写相应的客体

第五章 数据库完整性

- 数据库的完整性是指：
 - 数据的正确性
 - 是指数据是符合现实世界语义，反映了当前实际状况的
 - 数据的相容性
 - 是指数据库同一对象在不同关系表中的数据是符合逻辑的
- 为维护数据库的完整性，DBMS必须：
 1. 提供定义完整性约束条件的机制
 2. 提供完整性检查的方法
 3. 违约处理

数据库完整性

- 实体完整性：主码
- 参照完整性：外码
- 用户定义的完整性：NOT NULL、UNIQUE、CHECK、触发器

实体完整性检查和违约处理

- 插入或对主码列进行更新操作时，关系数据库管理系统按照实体完整性规则自动进行检查。包括：
 - 检查主码值是否唯一，如果不唯一则拒绝插入或修改
 - 检查主码的各个属性是否为空，只要有一个为空就拒绝插入或修改

外码（Foreign Key）定义

- 设 F 是基本关系 R 的一个或一组属性，但不是关系 R 的码。
如果 F 与基本关系 S 的主码 K_s 相对应，则称 F 是 R 的**外码**
- 基本关系 R 称为**参照关系**（Referencing Relation）
- 基本关系 S 称为**被参照关系**（Referenced Relation）
或**目标关系**（Target Relation）

[例] 学生实体、专业实体

学生（学号，姓名，性别，**专业号**，年龄）

专业（**专业号**，专业名）

参照完整性检查和违约处理

- 一个参照完整性将两个表中的相应元组联系起来
- 对被参照表和参照表进行增删改操作时有可能破坏参照完整性，必须进行检查

参照完整性检查和违约处理（续）

被参照表（例如Student）	参照表（例如SC）	违约处理
可能破坏参照完整性 ←	插入元组	拒绝
可能破坏参照完整性 ←	修改外码值	拒绝
删除元组 →	可能破坏参照完整性	拒绝/级连删除/设置为空值
修改主码值 →	可能破坏参照完整性	拒绝/级连修改/设置为空值

参照完整性检查和违约处理（续）

- 参照完整性违约处理

- （1）拒绝（**NO ACTION**）执行

- 不允许该操作执行。该策略一般设置为默认策略

- （2）级联（**CASCADE**）操作

- 当删除或修改被参照表（**Student**）的一个元组造成了与参照表（**SC**）的不一致，则删除或修改参照表中的所有造成不一致的元组

- （3）设置为空值（**SET-NULL**）

- 当删除或修改被参照表的一个元组时造成了不一致，则将参照表中的所有造成不一致的元组的对应属性设置为空值。

用户定义的完整性

1、属性上的约束条件

2、元组上的约束条件

第六章 关系数据理论

- 数据依赖分为函数依赖、多值依赖
- 平凡函数依赖与非平凡函数依赖
- 完全函数依赖与部分函数依赖
- 传递函数依赖
- 主码、候选码、全码、主属性、非主属性

第六章 关系数据理论

如果一个关系模式包含一些不合适的数据依赖，这样的关系模式会存在数据冗余、更新异常、插入异常、删除异常的问题。

规范化理论要讨论的内容：通过分解关系模式来消除其中不合适的数据依赖，以解决插入异常、删除异常、更新异常和数据冗余问题。

第一范式(1NF)

第二范式(2NF)

第三范式(3NF)

BCNF

Armstrong公理系统

- A1. 自反律 (Reflexivity) : 若 $Y \subseteq X \subseteq U$, 则 $X \rightarrow Y$ 为 F 所蕴含。
- A2. 增广律 (Augmentation) : 若 $X \rightarrow Y$ 为 F 所蕴含, 且 $Z \subseteq U$, 则 $XZ \rightarrow YZ$ 为 F 所蕴含。
- A3. 传递律 (Transitivity) : 若 $X \rightarrow Y$ 及 $Y \rightarrow Z$ 为 F 所蕴含, 则 $X \rightarrow Z$ 为 F 所蕴含。

推论

根据A1, A2, A3这三条推理规则可以得到下面三条推理规则:

- 合并规则: 由 $X \rightarrow Y$, $X \rightarrow Z$, 有 $X \rightarrow YZ$ 。
(A2, A3)
- 伪传递规则: 由 $X \rightarrow Y$, $WY \rightarrow Z$, 有 $XW \rightarrow Z$ 。
(A2, A3)
- 分解规则: 由 $X \rightarrow Y$ 及 $Z \subseteq Y$, 有 $X \rightarrow Z$ 。
(A1, A3)

函数依赖集的闭包是个无解问题

属性集 X 关于函数依赖集 F 的闭包，求取方法
作用：1) 判断是否为候选码；2) 某个函数依赖是否成立

($X \rightarrow Y$ 能由 F 根据Armstrong公理导出的充分必要条件是 $Y \subseteq X_F^+$)

数据依赖的公理系统（续）

- 求闭包的算法
- 算法6.1 求属性集 X ($X \subseteq U$) 关于 U 上的函数依赖集 F 的闭包 X_F^+
 - 输入: X, F
 - 输出: X_F^+
 - 步骤:

数据依赖的公理系统（续）

- ① 令 $X^{(0)}=X$, $i=0$
- ② 求 B , 这里 $B = \{ A \mid (\exists V)(\exists W)(V \rightarrow W \in F \wedge V \subseteq X^{(i)} \wedge A \in W) \}$ 。
- ③ $X^{(i+1)}=B \cup X^{(i)}$ 。
- ④ 判断 $X^{(i+1)}= X^{(i)}$ 。
- ⑤ 若 $X^{(i+1)}$ 与 $X^{(i)}$ 相等或 $X^{(i)}=U$, 则 $X^{(i)}$ 就是 X_F^+ , 算法终止。
- ⑥ 若否, 则 $i=i+1$, 返回第②步。

补充：候选码的求解算法

设关系模式 $R\langle U, F\rangle$

将R的所有属性分为 L、 R、 N和 LR四类，并令

L类: 仅出现在F的函数依赖左部的属性;

R类: 仅出现在F的函数依赖右部的属性;

N类: 在F的函数依赖左右两边都不出现的属性;

LR类: 在F的函数依赖左右两边都出现的属性。

候选码的求解算法(续)

算法：按以下步骤求候选键：

1. 只在F右部出现的属性（R类），不属于候选码；
2. 只在F左部出现的属性（L类），一定存在于某候选码当中；
3. 左右两边都不出现的属性（N类），一定存在于任何候选码当中；
4. 令X表示L、N两类中所有属性，求X的闭包，如果等于U，则X是候选码，算法结束。
5. 否则，将LR类中任取1个、2个、3个。。。与X进行任意组合K，求属性K的闭包，如果K的闭包等于U, 而且没有冗余属性，则K为候选码。

已知关系模式 $R\langle U, F \rangle$ ，其中

$U=\{A, B, C, D, E\}$;

$F=\{AB\rightarrow C, B\rightarrow D, C\rightarrow E, EC\rightarrow B, AC\rightarrow B\}$ 。

求候选码

L类型属性: A

R类型属性: D X

N类型属性: 无

LR类型属性: B、C、E

$(AB)^+=ABCDE$ $(AC)^+=ABCDE$ $(AE)^+=AE$

极小函数依赖集

- (1) 逐一检查 F 中各函数依赖 $FD_i: X \rightarrow Y$, 若 $Y = A_1 A_2 \dots A_k$, $k > 2$, 则用 $\{ X \rightarrow A_j \mid j=1, 2, \dots, k \}$ 来取代 $X \rightarrow Y$ 。
- (2) 逐一检查 F 中各函数依赖 $FD_i: X \rightarrow A$, 令 $G = F - \{X \rightarrow A\}$, 若 $A \in X_G^+$, 则从 F 中去掉此函数依赖。
- (3) 逐一取出 F 中各函数依赖 $FD_i: X \rightarrow A$, 设 $X = B_1 B_2 \dots B_m$, 逐一考查 B_i ($i=1, 2, \dots, m$), 若 $A \in (X - B_i)_F^+$, 则以 $X - B_i$ 取代 X 。

模式分解

三种模式分解等价的定义：

1. 分解具有无损连接性
2. 分解要保持函数依赖
3. 分解既要保持函数依赖，又要具有无损连接性

分解算法

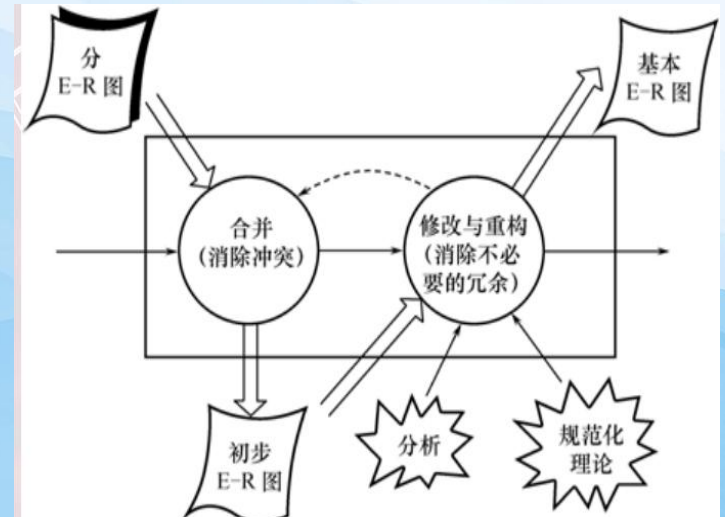
- 算法6.2（合成法）转换为3NF的保持函数依赖的分解
- 算法6.3 判别一个分解的无损连接性
- 算法6.4 转换为3NF既有无损连接性又保持函数依赖的分解
- 算法6.5（分解法）转换为BCNF的无损连接分解

第七章 数据库设计

- 需求分析：数据字典、数据流图
- 数据字典的内容
 - 数据项:数据项是不可再分的数据单位
 - 数据结构:数据结构反映了数据之间的组合关系。
 - 数据流:数据流是数据结构在系统内传输的路径。
 - 数据存储:数据存储是数据结构停留或保存的地方，也是数据流的来源和去向之一。
 - 处理过程:具体处理逻辑一般用判定表或判定树来描述

第七章 数据库设计

- 概念结构设计:**E-R图**(分E-R图,基本E-R图, 合并、消除冲突(属性冲突, 命名冲突, 结构冲突), 消除冗余)
 - 消除冗余: 在初步E-R图中可能存在冗余的数据和实体间冗余的联系
 - 所谓冗余的数据是指可由基本数据导出的数据, 冗余的联系是指可由其他联系导出的联系。冗余数据和冗余联系容易破坏数据库的完整性, 给数据库维护增加困难, 应当予以消除。



第七章 数据库设计

- 逻辑结构设计: 关系模式设计
 1. E-R图向关系模型的转换
 2. 数据模型的优化
 3. 设计用户子模式

第七章 数据库设计

- 数据库的物理设计:数据库在物理设备上的存储结构与存取方法
 - 数据库管理系统常用存取方法
 1. B+树索引存取方法
 2. Hash索引存取方法
 3. 聚簇存取方法
- 数据库实施和维护
 1. 数据的载入和应用程序的调试
 2. 数据库的试运行
 3. 数据库的运行和维护

第八章 MySQL程序设计

- 系统提供的数据类型
- 变量的使用方法
- 运算符和表达式的使用
- 函数的定义和使用方法
- 流程控制语句的使用
- 存储过程
- 触发器

存储过程和触发器

- 什么是存储过程
- 存储过程有什么优点
 - 游标使用
 - 存储过程定义
 - 常用流程控制语句

触发器

- 触发器是一种特殊的存储过程，常常用于实现强制业务规则和数据完整性，便于保护数据库的完整性。
- 触发器是自动的：触发器由系统自动执行，不能由应用程序调用，触发器是在用户对表的数据作了任何修改（比如手工输入或者应用程序采取的操作：**insert, update, delete**）之后立即被激活。
- 触发器在对表进行更新操作时（**UPDATE、INSERT 或 DELETE**）激活。也就是说触发器是建立在对表的更新操作之上的，一个表中可以建立多个触发器。

触发器

- 定义(insert、 delete、 update)
- 用到的表: new, old

第十章 数据库恢复技术

事务的基本概念

故障的种类

恢复的实现技术

恢复策略

1.事务

- 事务(Transaction)是用户定义的一个数据库操作序列，这些操作要么全做，要么全不做，是一个不可分割的工作单位。在关系数据库中，一个事务可以是一条SQL语句，一组SQL语句或整个程序。

2.事务的特性（ACID特性）

事务的ACID特性：

- 原子性（Atomicity）
- 一致性（Consistency）
- 隔离性（Isolation）
- 持续性（Durability）

故障的种类

数据库中会发生各种故障，可以分为以下几类：

1.事务内部的故障

2.系统故障

3.介质故障

4.计算机病毒

恢复的实现技术

恢复机制涉及的关键问题

1. 如何建立冗余数据

- 数据转储（backup）
- 登记日志文件（logging）

2. 如何利用这些冗余数据实施数据库恢复

恢复策略

当系统运行过程中发生故障，利用数据库后备副本和日志文件就可以将数据库恢复到故障前的某个一致性状态。不同故障其恢复策略和方法也不一样。

1 事务故障的恢复

2 系统故障的恢复

3 介质故障的恢复

第十一章 并发控制

- 并发控制机制的任务：对并发操作进行正确调度，保证事务的隔离性，保证数据库的一致性
 1. 并发操作带来的数据不一致性
 2. 封锁
 3. 封锁协议
 4. 活锁和死锁

并发操作带来的数据不一致性

❖ 并发操作带来的数据不一致性

1. 丢失修改

2. 脏读（**dirty read**）：也称读“脏”数据

3. 不可重复读

*4. 幻读（**phantom row**）

基本封锁类型

- 基本封锁类型

- 排它锁（**Exclusive Locks**，简记为**X锁**）

- 共享锁（**Share Locks**，简记为**S锁**）

保持数据一致性的常用封锁协议

- 三级封锁协议
 - 1.一级封锁协议
 - 2.二级封锁协议
 - 3.三级封锁协议

活锁和死锁

- 封锁技术可以有效地解决并行操作的一致性问题，但也带来一些新的问题
 - ✓活锁
 - ✓死锁

解决死锁的方法

两类方法

1. 死锁的预防
2. 死锁的诊断与解除

1. 死锁的预防

预防死锁的方法

(1) 一次封锁法

(2) 顺序封锁法

2. 死锁的诊断与解除

- 死锁的诊断

- (1) 超时法

- (2) 等待图法